

Bilan Semaine 4 — SQL basique & Mini-projet final

Outils : PostgreSQL · DBeaver

Bilan général

Ressenti global : SQL plus intuitif que Pandas, mais les réflexes sont encore à consolider.

La logique des requêtes est naturelle — elle suit une intention claire (quoi, d'où, comment filtrer, comment grouper).

Ce qui manque encore : l'automatisme. Le niveau de maîtrise Python acquis sur 3 semaines ne s'obtient pas en une semaine de SQL.

C'est normal et attendu. Le réflexe viendra avec les projets du mois 2.

Leçon fondamentale de la semaine

SQL et Pandas répondent aux mêmes questions avec des syntaxes différentes.

La logique mentale est identique — ce qui change, c'est l'ordre d'écriture et les mots-clés.

Maîtriser les deux en parallèle est un avantage réel sur le marché Data Analyst.

Ce qui a été fait

Mise en place de l'environnement

- Installation et configuration de PostgreSQL (version 18)
- Connexion DBeaver → PostgreSQL via localhost:5432
- Création de la base mois1_data
- Création des 3 tables à partir des CSV nettoyés (semaines 1 & 2)
- Import des données et vérification des COUNT(*) attendus
- Résolution du PATH Windows pour psql en terminal

Point de méthode identifié

PostgreSQL ≠ DBeaver : le serveur et l'interface sont deux choses séparées.

Les tables existent dans PostgreSQL, les fichiers .sql sont du texte sur le disque.

Modifier un fichier .sql ne touche pas la base de données — les deux sont indépendants.

Bloc 1 — SELECT, WHERE, ORDER BY, LIMIT

SELECT DISTINCT coffee_name FROM coffee_sales	
Ce que j'ai fait	Exploré les valeurs uniques de chaque colonne catégorielle sur les 3 datasets.
Pourquoi je l'ai fait	Comprendre ce que contient un dataset avant d'écrire des requêtes d'analyse — équivalent de .unique() en Pandas.
Ce que ça m'a appris	DISTINCT s'applique à la ligne entière, pas à une seule colonne. Combiné avec ORDER BY, il donne un inventaire propre et trié.

SELECT ... FROM coffee_sales WHERE money BETWEEN 30 AND 40	
Ce que j'ai fait	Filtré les transactions par plage de valeurs avec WHERE et BETWEEN.
Pourquoi je l'ai fait	Retransposer les filtres Pandas (df[df['money'].between(30, 40)]) en SQL — même logique, syntaxe différente.
Ce que ça m'a appris	BETWEEN est inclusif aux deux bornes. Pour exclure une borne, utiliser > et < séparément.

Bloc 2 — GROUP BY et agrégats

GROUP BY coffee_name + SUM / COUNT / AVG / ROUND	
Ce que j'ai fait	Calculé le CA, le nombre de ventes et le prix moyen par produit sur coffee_sales. Appliqué la même logique sur Transilien (par gare et par type de jour).
Pourquoi je l'ai fait	Retransposer les groupby() Pandas de la semaine 3 — vérifier que les résultats SQL correspondent exactement aux résultats Pandas.
Ce que ça m'a appris	Les résultats sont identiques. GROUP BY est la traduction directe de groupby(). La règle absolue : toute colonne dans SELECT qui n'est pas une agrégation doit être dans GROUP BY.

HAVING COUNT(*) > 100	
Ce que j'ai fait	Filtré les produits ayant plus de 100 ventes, après regroupement.
Pourquoi je l'ai fait	Comprendre la distinction entre WHERE (filtre avant groupement) et HAVING (filtre après groupement).
Ce que ça m'a appris	WHERE ne peut pas accéder aux résultats d'agrégation. HAVING est indispensable pour filtrer sur COUNT(*), SUM(), AVG(). C'est un piège classique et récurrent.

EXTRACT(MONTH FROM date) — TO_CHAR(date, 'Month')	
Ce que j'ai fait	Calculé le CA par mois et analysé le profil de clientèle par jour de la semaine.
Pourquoi je l'ai fait	Reproduire les extractions .dt.month et .dt.day_name() préparées en semaine 2 — les mêmes colonnes extraites en Pandas existent aussi nativement en SQL.

Ce que ça m'a appris	EXTRACT() retourne un entier (utile pour trier). TO_CHAR() retourne du texte (utile pour afficher). Les deux sont complémentaires selon l'usage.
-----------------------------	--

Bloc 3 — Jointures

JOIN coffee_catalogue AS cc ON cs.coffee_name = cc.coffee_name	
Ce que j'ai fait	Crée une table catalogue artificielle (catégorie, prix catalogue) et joint avec coffee_sales pour croiser les données.
Pourquoi je l'ai fait	Les datasets du mois 1 sont mono-table — créer un cas pratique concret pour comprendre la logique JOIN avant de la rencontrer en conditions réelles au mois 2.
Ce que ça m'a appris	Un JOIN relie deux tables sur une colonne commune. Sans correspondance exacte (même casse, même valeur), les lignes sont exclues du résultat. Hot Chocolate était absent du catalogue initial — diagnostiqué et corrigé.

Mini-projet final — Coffee Sales en SQL pur

3 questions métier traitées sans Pandas :

Question	Résultat clé
Quels produits sont les plus rentables ?	Latte domine le CA — Americano with Milk domine le volume. Même conclusion qu'en semaine 3.
Comment évoluent les ventes dans le temps ?	Analyse mois par mois avec EXTRACT + TO_CHAR pour affichage lisible.
Quel profil de clientèle selon le jour ?	Dimanche = jour le plus faible. Semaine = clientèle principale.

Notion nouvelle rencontrée : window function

SUM(SUM(money)) OVER () — calcule le total général sans GROUP BY supplémentaire. Utilisée pour calculer la part de CA de chaque produit (part_ca_pct). À retenir comme outil existant — la maîtrise complète viendra plus tard.

Organisation et GitHub

- Dossier sql/ créé dans data-foundations-month1/
- schema.sql : les 3 CREATE TABLE réunis
- coffee_sales_queries.sql : blocs 1, 2, 3 + mini-projet
- transilien_queries.sql : blocs 1 et 2
- tmdb_queries.sql : blocs 1 et 2
- README mis à jour : semaine 4 + PostgreSQL/DBEaver dans les outils
- Commits effectués via GitHub Desktop

Table de correspondance Pandas ↔ SQL

Pandas	SQL
pd.read_csv()	COPY table FROM 'fichier.csv' CSV HEADER;
df.head(10)	SELECT * FROM table LIMIT 10;
df['col'].unique()	SELECT DISTINCT col FROM table;
df[df['col'] > x]	WHERE col > x
df.groupby('col').sum()	GROUP BY col + SUM(col)
sort_values('col', ascending=False)	ORDER BY col DESC
df.nlargest(5, 'col')	ORDER BY col DESC LIMIT 5
filtre post-groupby	HAVING
.dt.month	EXTRACT(MONTH FROM date)
.dt.day_name()	TO_CHAR(date, 'Day')

Points encore fragiles

Réflexe WHERE vs HAVING

La distinction est comprise intellectuellement mais pas encore automatique à l'écriture. Le réflexe viendra avec la répétition — notamment sur les projets mois 2 où l'erreur sera faite et corrigée en conditions réelles.

Ordre d'écriture vs ordre d'exécution

Écriture : SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT

Exécution : FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

L'ordre d'exécution explique pourquoi les alias définis dans SELECT ne sont pas disponibles dans WHERE — SELECT n'a pas encore été évalué à ce stade.

Automatisme des guillemets doubles sur Transilien

Les colonnes avec espaces ("Nom Gare", "Code Gare", "Somme de Montants"...) nécessitent des guillemets doubles obligatoires en PostgreSQL. Oubli = erreur de syntaxe. À intégrer comme réflexe sur tout dataset avec des noms de colonnes non standards.

Window functions

SUM(SUM(money)) OVER () a été utilisée dans le mini-projet mais la logique complète (PARTITION BY, ORDER BY dans OVER) n'a pas été explorée. Ce sera un point du mois 2 ou 3.

Ce qui reste à voir

SQL — à approfondir au mois 2

- Sous-requêtes (subqueries) — requêtes imbriquées dans WHERE ou SELECT
- CTEs (Common Table Expressions) — WITH ... AS — alternative lisible aux sous-requêtes
- Window functions complètes — PARTITION BY, ROW_NUMBER(), RANK(), LAG(), LEAD()
- Jointures multiples — joindre plus de deux tables en une seule requête
- LEFT JOIN en conditions réelles — sur des datasets avec vraies relations entre tables
- Indexes et performance — notions de base pour comprendre pourquoi une requête est lente

PostgreSQL / DBeaver — à consolider

- Exporter les résultats de requêtes en CSV directement depuis DBeaver
- Comprendre les types de données PostgreSQL avancés (JSONB, ARRAY, etc.) — aperçu théorique
- Vues (CREATE VIEW) — enregistrer une requête comme une table virtuelle réutilisable

Mois 2 — ce qui arrive

Prochain objectif

Projet Data Analyst complet sur un dataset e-commerce ou marketing.

SQL et Pandas utilisés en parallèle sur le même projet — consolidation des deux.

Points de vigilance identifiés

Situation	Ce qu'il faut faire
Filtrer sur un agrégat	Utiliser HAVING, jamais WHERE
Colonne avec espace dans le nom	Entourer de guillemets doubles : "Nom Gare"
Alias dans WHERE	Impossible — répéter l'expression ou utiliser HAVING
Créer une table dans la mauvaise base	Vérifier la base active en haut de l'éditeur SQL DBeaver
Chercher les doublons avec une colonne d'index	Supprimer la colonne d'index AVANT — même réflexe qu'en Pandas semaine 2

Semaine 4 terminée — Mois 1 bouclé.

SQL : intuition acquise, réflexes en construction. Le mois 2 consolidera les deux.